

TECHNO INDIA COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

Computer Network Lab Manual

Paper Code: CS692



Objective: 1. Understand the concept of layering/encapsulation by looking at Link, IP and TCP headers.
2. Understand the concept of multiplexing using Ethernet "frame type" field, IP "protocol field", transport "port number" field.

LAB 5

1. Understanding how the host manages network information with the following files
/etc/hostname; /etc/hosts; /etc/network/interfaces; /etc/resolv.conf; /etc/protocols; /etc/services
2. Packet capturing using tcpdump & Wireshark & demonstrating the concept of encapsulation of frames, packets, segments and messages

3
Hours

Experiment1: Play around with tcpdump, wireshark, ping, arp, route, ifconfig, host, netstat etc.

Look at */etc/hostname*; */etc/hosts*; */etc/network/interfaces*; */etc/resolv.conf*;
/etc/protocols; */etc/services* and understand what the files are for.

At the end of this exercise, you should have some basic understanding of how a host manages network information as well as gain some experience on using networking tools.

You should be able to collect a trace (write to a file) via tcpdump and view the trace in wireshark (using the -r option)

Simple Stuff: Answer the following

1. What is your machine's host name and IP address? How did you get this information?
2. What is the next hop router's IP address and MAC address? How did you get this information?
3. What is the local DNS server's host name and IP address? How did you get this information?
4. What do the numbers in the file */etc/protocols* represent?
5. What is the port number associated with applications: ssh, ftp, nfs, smtp (email)?

Experiment 2: Encapsulation and Demultiplexing:

Goal: To understand layering and demultiplexing, Mr. X wants to capture packets. He also wants to understand how web flows operate at the same time. So, help him design an experiment that captures only those packets that are exchanged between his machine and TICT web server when he clicks the url <http://tictcollege.in>

Guidance:

1. Run tcpdump with -n option to avoid name lookup.
2. Use wget (command: **wget http://tictcollege.in** or <http://csemoodle.tict.edu.in> or <http://192.168.10.5>) to download the url. You could also use firefox/chrome, but this is cleaner and simpler.
3. Your trace should not capture any background traffic.
4. Before answering the questions, explore different packets by clicking on the individual packets. Also note the sequence of packet exchange.

Report:

1. Explain your experimental design by specifying the exact commands (with options) you will run and in which order. Avoid description unless absolutely necessary.
2. Select the first TCP packet listed.
 - a) Which next-hop node is it destined to? Specify the next-hop node's MAC and IP address.
How did you determine this information?
 - b) Who is the packet's final destination? Specify the final destinations' MAC and IP address?
How did you determine this information?
 - c) What are the fields used at the link (Ethernet), IP and TCP headers to demux the packet at the destination? Specify the values of these fields in decimal format and the corresponding process (protocol) the packet is passed to.
3. Apart from the above reporting, name your trace file as "lab2experiment2.out" and add the file to your roll number directory.

/etc/hostname & /etc/hosts

The `/etc/hostname` file should contain the hostname only

The `/etc/hosts` file is a mapping of hosts to IP addresses and traditionally has used hostname only as the primary name.

So what is a host name and a FQDN? The hostname is the name of a host. Maybe it your help to define a host. In this instance a host is the specific machine. If we had a machine named ***oak*** in the DNS domain of ***trees.com*** we would have this

Code:

```
host = oak
FQDN = oak.trees.com
```

The contents of the `/etc/hostname` file would look like this

Code:

```
oak
```

The `etc/hosts` file is a little different. It is the primitive of DNS naming. The mapping of IP addresses to *hostnames* (not FQDN) in the beginning. It would look like this

Code:

```
127.0.0.1      localhost
192.168.1.2    oak oak.trees.com
192.168.1.3    maple maple.trees.com
```

/etc/network/interfaces

`/etc/network/interfaces` file contains network interface configuration information for the both Ubuntu and Debian Linux. This is where you configure how your system is connected to the network.

Setup interface to dhcp

To setup `eth0` to dhcp, enter:

```
auto eth0
iface eth0 inet dhcp
```

For static ip

```
auto eth0
iface eth0 inet static
address 192.168.1.5
```

```
netmask 255.255.255.0
gateway 192.168.1.254
dns-nameservers 192.168.1.254
```

File located at </usr/share/doc/ifupdown/examples/network-interfaces>, use this file as reference

/etc/resolv.conf

The *resolv.conf* [configuration file](#) contains information that determines the operational parameters of the DNS resolver. The DNS resolver allows applications running in the operating system to translate human-friendly [domain names](#) into the numeric [IP addresses](#) that are required for access to resources on the [local area network](#) or the [Internet](#). The process of determining IP addresses from domain names is called *resolving*.

The */etc/resolv.conf* file is used to provide specific information about the DNS configuration for the local machine.

- It contains a list of Domains to search
- Strict order of name servers to use

It also contains a list of IP addresses of [nameservers](#) available for resolution. An example file is:

```
search example.com local.lan
nameserver 172.16.1.254
nameserver 172.16.2.254
```

resolv.conf is usually located in the [/etc](#) directory of the [file system](#). The file is either maintained manually, or when [DHCP](#) is used, it is usually updated with the utility [resolvconf](#).

/etc/protocols

The */etc/protocols* file contains information regarding the known protocols used in the DARPA Internet

A Sample /etc/protocols File

```
#
# Internet (IP) protocols
#
ip          0          IP          # internet protocol, pseudo protocol number
icmp        1          ICMP        # internet control message protocol
igmp        2          IGMP        # internet group multicast protocol
tcp         6          TCP         # transmission control protocol
udp         17         UDP         # user datagram protocol
```

```
raw      255      RAW          # RAW IP interface
```

/etc/services

The port numbers on which certain “standard” services are offered are defined in the Assigned Numbers RFC. To enable server and client programs to convert service names to these numbers, at least part of the list is kept on each host; it is stored in a file called `/etc/services`. An entry is made up like this:

```
service port/protocol  [aliases]
```

Here, *service* specifies the service name, *port* defines the port the service is offered on, and *protocol* defines which transport protocol is used. Commonly, the latter field is either *udp* or *tcp*. It is possible for a service to be offered for more than one protocol, as well as offering different services on the same port as long as the protocols are different. The *aliases* field allows you to specify alternative names for the same service.

A Sample /etc/services File

```
# The services file:
#
# well-known services
echo      7/tcp      # Echo
echo      7/udp      #
discard   9/tcp      sink null  # Discard
discard   9/udp      sink null  #
daytime   13/tcp     # Daytime
daytime   13/udp     #
chargen   19/tcp     ttytst source # Character Generator
chargen   19/udp     ttytst source #
ftp-data  20/tcp     # File Transfer Protocol (Data)
ftp       21/tcp     # File Transfer Protocol (Control)
telnet    23/tcp     # Virtual Terminal Protocol
smtp      25/tcp     # Simple Mail Transfer Protocol
nntp      119/tcp    readnews    # Network News Transfer Protocol
#
# UNIX services
exec      512/tcp    # BSD rexecd
biff      512/udp    comsat      # mail notification
login     513/tcp    # remote login
who       513/udp    whod        # remote who and uptime
shell     514/tcp    cmd         # remote command, no passwd used
syslog    514/udp    # remote system logging
printer   515/tcp    spooler     # remote print spooling
route     520/udp    router routed # routing information protocol
```

Wireshark

Wireshark is a [free and open-source packet analyzer](#). It is used for [network](#) troubleshooting, analysis, software and [communications protocol](#) development, and education. Originally named **Ethereal**, the project was renamed Wireshark in May 2006. Wireshark is very similar to [tcpdump](#), but has a [graphical front-end](#), plus some integrated sorting and filtering options.

Wireshark lets the user put [network interface controllers](#) that support [promiscuous mode](#) into that mode, so they can see all traffic visible on that interface, not just traffic addressed to one of the interface's configured addresses and broadcast/multicast traffic. However, when capturing with a [packet analyzer](#) in promiscuous mode on a port on a [network switch](#), not all traffic through the switch is necessarily sent to the port where the capture is done, so capturing in promiscuous mode is not necessarily sufficient to see all network traffic

Install wireshark in Ubuntu

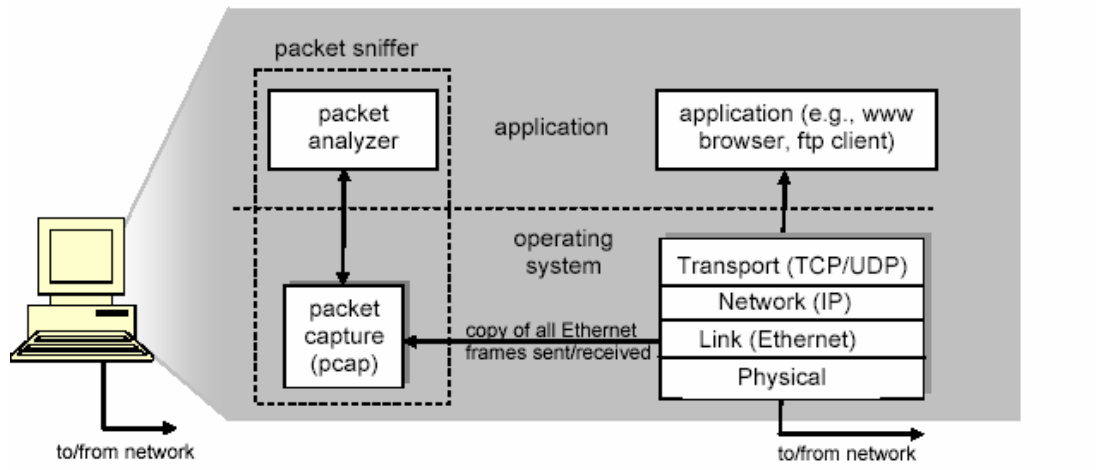
```
sudo apt-get install wireshark
```

PACKER SNIFFER

The basic tool for observing the messages exchanged between executing protocol entities is called a **packet sniffer**. As the name suggests, a packet sniffer captures (“sniffs”) messages being sent/received from/by your computer; it will also typically store and/or display the contents of the various protocol fields in these captured messages. A packet sniffer itself is passive. It observes messages being sent and received by applications and protocols running on your computer, but never sends packets itself. Similarly, received packets are never explicitly addressed to the packet sniffer. Instead, a packet sniffer receives a *copy* of packets that are sent / received from/by application and protocols executing on your machine.

Figure 1 shows the structure of a packet sniffer. At the right of Figure 1 are the protocols (in this case, Internet protocols) and applications (such as a web browser or ftp client) that normally run on your computer. The packet sniffer, shown within the dashed rectangle in Figure 1 is an addition to the usual software in your computer, and consists of two parts. The **packet capture library** receives a copy of every link-layer frame that is sent from or received by your computer. Messages exchanged by higher layer protocols such as HTTP, FTP, TCP, UDP, DNS, or IP all are eventually encapsulated in link-layer frames that are transmitted over physical media such as an Ethernet cable. In Figure 1, the assumed physical media is an Ethernet, and so all upper layer protocols are eventually encapsulated within an Ethernet frame. Capturing all link-layer frames thus gives you all messages sent/received from/by all protocols and applications executing in your computer.

The second component of a packet sniffer is the **packet analyzer**, which displays the contents of all fields within a protocol message. In order to do so, the packet analyzer must “understand” the structure of all messages exchanged by protocols. For example, suppose we are interested in displaying the various fields in messages exchanged by the HTTP protocol in Figure 1. The packet analyzer understands the format of Ethernet frames, and so can identify the IP datagram within an Ethernet frame. It also understands the IP datagram format, so that it can extract the TCP segment within the IP datagram. Finally, it understands the TCP segment structure, so it can extract the HTTP message contained in the TCP segment. Finally, it understands the HTTP protocol and so, for example, knows that the first bytes of an HTTP message will contain the string “GET,” “POST,” or “HEAD”.



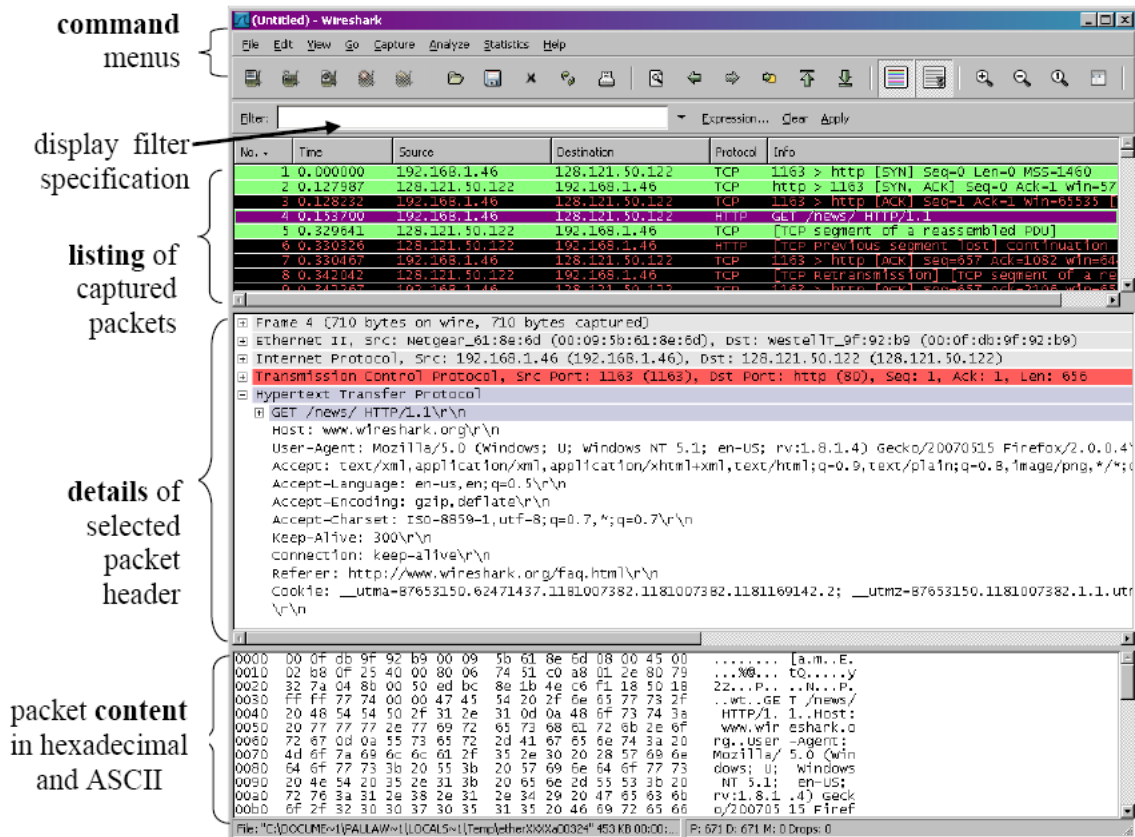
We will be using the Wireshark packet sniffer [<http://www.wireshark.org/>] for these labs, allowing us to display the contents of messages being sent/received from/by protocols at different levels of the protocol stack. (Technically speaking, Wireshark is a packet analyzer that uses a packet capture library in your computer). Wireshark is a free network protocol analyzer that runs on Windows, Linux/Unix, and Mac computers. It's an ideal packet analyzer for our labs – it is stable, has a large user base and well-documented support that includes a user-guide (http://www.wireshark.org/docs/wsug_html_chunked/), manpages (<http://www.wireshark.org/docs/man-pages/>), and a detailed FAQ (<http://www.wireshark.org/faq.html>), rich functionality that includes the capability to analyze hundreds of protocols, and a well-designed user interface. It operates in computers using Ethernet, Token-Ring, FDDI, serial (PPP and SLIP), 802.11 wireless LANs, and ATM connections (if the OS on which it's running allows Wireshark to do so).

Running Wireshark

When you run the Wireshark program, the Wireshark graphical user interface shown in Figure 2 will be displayed. Initially, no data will be displayed in the various windows.

The Wireshark interface has five major components:

1. The **command menus** are standard pulldown menus located at the top of the window. Of interest to us now are the File and Capture menus. The File menu allows you to save captured packet data or open a file containing previously captured packet data, and exit the Wireshark application. The Capture menu allows you to begin packet capture.
2. The **packet-listing window** displays a one-line summary for each packet captured, including the packet number (assigned by Wireshark; this is *not* a packet number contained in any protocol's header), the time at which the packet was captured, the packet's source and destination addresses, the protocol type, and protocol-specific information contained in the packet. The packet listing can be sorted according to any of these categories by clicking on a column name. The protocol type field lists the highest level protocol that sent or received this packet, i.e., the protocol that is the source or ultimate sink for this packet.
3. The **packet-header details window** provides details about the packet selected (highlighted) in the packet listing window. (To select a packet in the packet listing window, place the cursor over the packet's one-line summary in the packet listing window and click with the left mouse button.). These details include information about the Ethernet frame and IP datagram that contains this packet. The amount of Ethernet and IP-layer detail displayed can be expanded or minimized by clicking on the right-pointing or down-pointing arrowhead to the left of the Ethernet frame or IP datagram line in the packet details window.



If the packet has been carried over TCP or UDP, TCP or UDP details will also be displayed, which can similarly be expanded or minimized. Finally, details about the highest level protocol that sent or received this packet are also provided.

- The **packet-contents window** displays the entire contents of the captured frame, in both ASCII and hexadecimal format.
- Towards the top of the Wireshark graphical user interface, is the **packet display filter field**, into which a protocol name or other information can be entered in order to filter the information displayed in the packet-listing window (and hence the packet-header and packet-contents windows). In the example below, we'll use the packet-display filter field to have Wireshark hide (not display) packets except those that correspond to HTTP messages.